V. Sarcar, *Java Design Patterns*
https://doi.org/10.1007/978-1-4842-7971-7_25

25. Interpreter Pattern

Vaskaran Sarcar[1]
(1) Garia, Kolkata, India

This chapter covers the Interpreter pattern.

GoF Definition

Given a language, it defines a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.

Concept

This pattern plays the role of a translator, and it is often used to *evaluate sentences in a language*. So, you first need to define grammar to represent the language. Then the interpreter deals with that grammar. **This pattern works best if the grammar is simple.**

Points to Note
To understand this pattern, it's helpful if you are familiar with some key terms like words (or sentences), grammar, and languages in Automata, which is a big topic. A detailed discussion of this is beyond the scope of this book. For now, you simply need to know that in the formal language, an alphabet may contain an infinite number of elements, a word can be a finite sequence of letters (simply strings), and a set of all strings generated by a grammar G is called the language generated the grammar G. Normally a grammar is represented by a tuple (V, T, S, P) where V is a set of non-terminal symbols, T is a set of terminal symbols, S is the start symbol, and P is the production rules.

Consider an example. Suppose you have a grammar G=(V,T,S,P) where

V={S},

T={a,b},

P={S->aSbS,S->bSaS,S->ε },

S={S};

The ε in this grammar denotes an empty string. The production rules tell you that you can substitute an S with any of aSbS, bSaS, or an empty string (ε).

On further investigation, you understand that this grammar can generate an equal number of a's and b's like ab, ba, abab, baab, and so forth. But

how? The following steps show a derivation process of getting abba from this grammar. Notice that each time you substitute the leftmost variable in this example:

S

aSbS [since S->aSbS]

abS [since S->ε]

ab**bSaS** [since S->bSaS]

abbaS [since S->ε]

abba [since S->ε]

In the same way, you can generate baab from this grammar. Here are the derivation steps for another quick reference:

S

bSaS [since S->bSaS]

baS [since S->ε]

ba**aSbS** [since S->aSbS]

baabS [since S->ε]

baab [since S->ε]

Each class in this pattern represents a rule in that language and it should have a method to interpret an expression. So, to handle a greater number of rules, you create a greater number of classes, and this is why the Interpreter pattern is seldom used to handle very complex grammar.

A typical structure of this pattern is often described with a diagram similar to Figure 25-1.

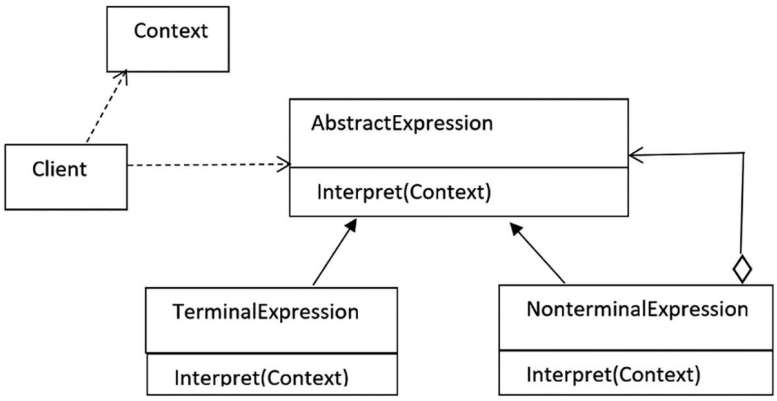

Figure 25-1 Structure of a typical Interpreter pattern

The terms are described as follows:

- `AbstractExpression`: Typically an interface with an `interpret` method. You need to pass a context object to this method.

- `TerminalExpression`: It is used for terminal expressions. A terminal expression does not need other expressions to interpret. They are basically leaf nodes (i.e., they do not have child nodes) in the data structure.
- `NonterminalExpression`: It is used for non-terminal expressions. It's also termed as `AlternationExpression`, `RepititionExpression`, or `SequenceExpression`. They are like composites that can contain both terminal and nonterminal expressions. When you call the `interpret` method on this, you basically call `interpret` on all of its children. To make this clear in your mind, consider a simple arithmetic expression such as `5-3`. This is a non-terminal expression where 5 and 3 are the terminal expressions with values 5 and 3, respectively. The minus (-) operator with the operands (the terminal expressions) form a non-terminal expression.
- **Context**: It holds the global information that the interpreter needs.
- **Client**: It calls the `interpret()` method. It can optionally build the syntax tree based on the rules of the language.

Points to Remember

It is already mentioned that an interpreter is used to process a language with simple rules or grammar. Ideally, developers do not want to create their own languages. This is why they seldom use this pattern.

Real-Life Example

Think of a translator who translates a foreign language. You can also consider music notes as grammar, where musicians play the role of interpreters.

Computer World Example

The Java compiler interprets the Java source code into bytecode that is understandable by the Java Virtual Machine. In C#, the source code is converted to Microsoft Intermediate Language (MSIL) code, which is interpreted by the Common Language Runtime. Upon execution, this MSIL is converted to native code (binary executable code) by the Just-In-Time compiler. Here is another example for you:

- If you notice the `java.util.Format` class, you will learn that this abstract class is used for formatting locale-sensitive information such as dates, messages, and numbers. So, any subclass of it can be considered to use this pattern.

Implementation

In demonstration 1, you see an example to interpret a Boolean expression. In other words, the Interpreter pattern is being used as a rule validator.

To implement this, start by defining simple grammar. The constants `true` and `false` are the terminals in this grammar and they are used to get the final value of an output. Nonterminal symbols are the expressions that contain the following operators: `and`, `not`, and `or`. A client may want to know the result of a simple expression such as `emp1` or they may be interested to know the value of a complex expression such as `emp1 and emp2 or emp3 or emp4`. So, `IndividualEmployee` evaluates a

variable in a context and assigns a value (true or false) to each variable (`emp1,emp2,emp3 or emp4`). Here is the grammar for you:

```
Employee ::= IndividualEmployee| Constant | OrExp | AndExp | NotExp
AndExp ::= Employee 'and' Employee
OrExp ::= Employee 'or' Employee
NotExp ::= 'not' Employee
Constant ::= 'true' | 'false'
IndividualEmployee ::= 'emp1' | 'emp2' | 'emp3' | 'emp4'
```

The following steps show a derivation process of getting `emp1 or emp2` from this grammar (the transitions are in bold and a different font):

```
Employee
OrExp [since Employee->OrExp]
Employee 'or' Employee [since OrExp-> Employee 'or' Employee]
IndividualEmployee 'or' Employee [since Employee-> IndividualEmployee]
IndividualEmployee 'or' IndividualEmployee [since Employee->
                                            IndividualEmployee]
emp1 or IndividualEmployee [since IndividualEmployee ->emp1]
emp1 or emp2 [since IndividualEmployee ->emp2]
```

Similarly, you can derive `emp2 and emp4` from this grammar as follows:

```
Employee
AndExp [since Employee->AndExp]
Employee 'and' Employee [since AndExp-> Employee 'and' Employee]
IndividualEmployee 'and' Employee [since Employee-> IndividualEmployee]
IndividualEmployee 'and' IndividualEmployee [since Employee-> IndividualEmployee]
emp2 and IndividualEmployee [since Employee ->emp2]
emp2 and emp4 [since IndividualEmployee ->emp4]
```

A client may want to know the final value of such an expression. So, in this application, you will evaluate that.

Now let's build the application. There are some important steps (which are followed in this example) to consider before you implement this pattern. They are as follows:

- Step 1: Define the rules of the language for which you want to build an interpreter.
- Step 2: Define an abstract class or an interface to represent an expression. It should contain a method to interpret an expression.
- Step 2A: Identify terminal, non-terminal expressions, and variables. For example, in the upcoming example, the constants `true` and `false` are the terminal symbols, which are Boolean variables.
- Step2B: Create non-terminal expression classes. Each of them calls the `interpret(...)` method on their children. For example, in the upcoming example, the `OrExpression` and `AndExpression` classes are non-terminal expression classes. In demonstration 1, you'll see that these classes implement the interface `Employee`, which has the `interpret(...)` method
- Step2C: `IndividualEmployee` is used to evaluate a variable based on the current context. So, this class also implements the `Employee` interface and provides an implementation for the interface method.
- Step 3: Build the abstract syntax tree using these classes. You can do this inside the client code or you can create a separate class to accomplish the task.

- Step 4: The client now uses this tree to interpret a sentence.
- Step 5: Pass the context to the interpreter. It typically will have the sentences that are to be interpreted. An interpreter can also perform additional tasks using this context.

Here you instantiate different employees with their "year of experience" and current grades. For simplicity, there are four employees with four different grades: G1, G2, G3, and G4. Notice the following lines:

```
        Employee emp1 = new IndividualEmployee(5,"G1");
        Employee emp2 = new IndividualEmployee(10,"G2");
        Employee emp3 = new IndividualEmployee(15,"G3");
        Employee emp4 = new IndividualEmployee(20,"G4");
```

In this example, you want to validate some conditions against the context that says to get a promotion, an employee should have a minimum of 10 years of experience and they should be either from G2 grade or G3 grade. Once these expressions are interpreted, you will see the output in terms of a Boolean value. This is why you see the following code snippet:

```
// Minimum Criteria for promotion is:
// The year of experience is a minimum of 10 yrs. and
// Employee grade should be either G2 or G3
Context context=new Context(10,"G2","G3");
```

You have just seen that minimum experience in years and the allowed grades (for promotion) are passed to the `Context` class constructor. So, the following segment of code in the `Context` class should make sense to you:

```
private int yearofExperience;
private List<String> permissibleGrades;
public Context(int experience, String... allowedGrades) {
     this.yearofExperience = experience;
     this.permissibleGrades = new ArrayList<>();
     for (String grade : allowedGrades) {
     permissibleGrades.add(grade);
 }
}
```

In the upcoming example, you validate three different types of expressions:

- Case 1: Whether an employee is eligible for the promotion. You may need to evaluate a simple expression like `emp1`. In the client code, you use the method `validateSingleEmployee()` for this purpose.
- Case 2: Whether a combination such as `emp1 or emp2, emp1 and emp2`, or `! emp3` is eligible for the promotion. This is why you see the `validateExpression(...)` method inside the `ExpressionBuilder` class. In the client code, you use the method `validateSimpleRules()` for this purpose.
- Case 3: Finding the value of a complex expression like `emp1 and (emp2 or (emp3 or emp4))`. This is why you see the `validateComplexExpression(...)` method inside the `ExpressionBuilder` class. In the client code, you use the method `validateComplexRules()` for this purpose.

You understand that evaluating case 1 is straightforward. Evaluating case 2 is also not tough. Notice that in all these cases, the value of an expres-

sion changes if the context changes. For example, the expression `emp1 or emp3` results in `true` now . But, to get a promotion, if you change the minimum experience requirement from 10 to 16 years, then neither `emp1` nor `emp3` meets the criteria. As a result, the value of the expression- `emp1 or emp3` will become `false`.

Evaluating case 3 is not very easy. You cannot predict the value instantly. You use an optional utility class called `ExpressionBuilder`, which has two utility methods, `validateExpression` and `validateComplexExpression`. The first one is used to evaluate the expressions from case 2 and the other one is used to evaluate an expression from case 3. You follow a step-by-step approach to get the final value of the expression. Refer to the associated comments for your easy understanding. You can apply the same technique to evaluate other complex expressions. When I evaluate a complex expression, I name different `Employee` variables as `firstPhase`, `secondPhase`, and so on to help you understand the step-by-step approach of the method. Here is a sample code:

```
// Validating the rule: emp1 and (emp2 or (emp3 or emp4))
public Employee validateComplexExpression(Employee emp1, Employee emp2, Employee emp3



 // (emp3 or emp4)
 Employee firstPhase = new OrExpression(emp3, emp4);
 // emp2 or (emp3 or emp4)
 Employee secondPhase = new OrExpression(emp2, firstPhase);
 // emp1 and (emp2 or (emp3 or emp4))
 Employee finalPhase = new AndExpression(emp1, secondPhase);
 return finalPhase;
}
```

Point to Note

One important point to note is that this design pattern does not instruct you on how to build the syntax tree or how to parse the sentences. It gives you the freedom of choose how to proceed. So, to present a simple scenario and keep things easy, I use class `EmployeeBuilder` with two methods to accomplish the task. For me, it makes sense to put these methods in a common place. But I acknowledge the fact that I violate the SRP principle here because I include methods for validating simple rules as well as complex rules. So, if you have a concern for the SRP, you can refactor the code. I leave the task for you.

Now let's see other parts of the program. The `Employee` is an interface with the `interpret(...)` method as follows:

```
interface Employee {
   public boolean interpret(Context context);
}
```

The `IndividualEmployee` class has the following constructor:

```
public IndividualEmployee(int experience, String grade) {
     this.experience = experience;
     this.grade = grade;
}
```

I already mentioned that to get a promotion, an employee should have minimum of 10 years of experience and they should be either from G2 grade or G3 grade. You pass this information inside the `Context` object. So, this class implements `Employee` interface method as follows:

```
   @Override
public boolean interpret(Context context) {
 if (experience >= context.getYearofExperience() &&
           context.getPermissibleGrades().contains(grade)) {
       return true;
     }
   return false;
}
```

The `OrExpression` is a non-terminal class and it implements `Employee` interface method as follows:

```
@Override
public boolean interpret(Context context) {
 return emp1.interpret(context) || emp2.interpret(context);
}
```

Similarly, `AndExpression` implements the `Employee` interface method as follows:

```
@Override
public boolean interpret(Context context) {
 return emp1.interpret(context) && emp2.interpret(context);
}
```

The `NotExpression` is another non-terminal class that implements the `Employee` interface method as follows:

```
@Override
public boolean interpret(Context context) {
  return !emp.interpret(context);
}
```

Lastly, there is an `InvalidExpression` class. It's used for the sake of completeness only. The `interpret()` method of this class always returns false. The remaining code is easy to understand, so continue reading.

Class Diagram

Figure 25-2 shows the class diagram. To make the diagram short, I show the attributes of the `Context` class only. You can see the other methods and fields from the Package Explorer view and the complete demonstration code.

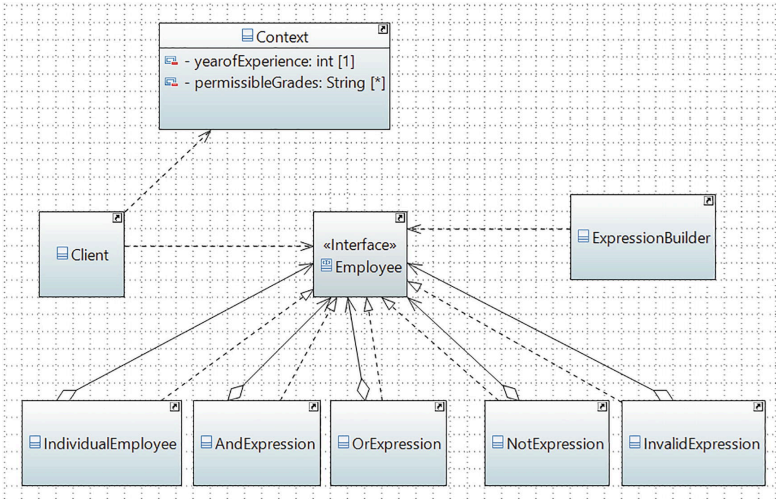

Figure 25-2 Class diagram

Package Explorer View

Figure 25-3 shows the high-level structure of the program. It is a big snapshot and there are many parts to this program. To display the important parts of the program, I expand some selected portions for your reference.

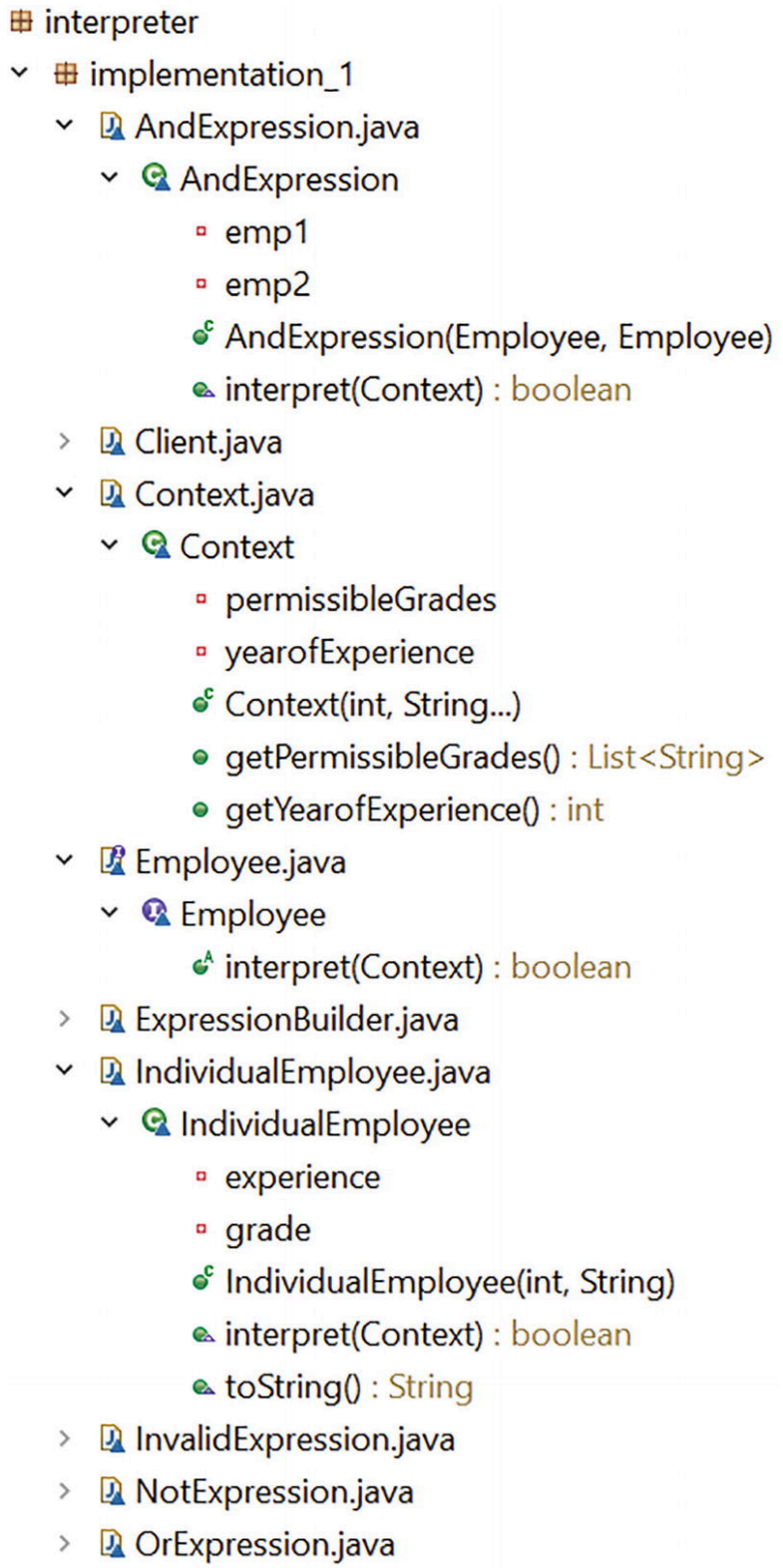

Figure 25-3 Package Explorer view

Demonstration 1

Here's the complete demonstration. All parts of the program are inside the package `jdp3e.interpreter.implementation_1`.

```
// Employee.java
interface Employee {
     public boolean interpret(Context context);
}
// IndividualEmployee.java
class IndividualEmployee implements Employee {
```

```
     private int experience;
     private String grade;
     public IndividualEmployee(int experience, String grade) {
          this.experience = experience;
          this.grade = grade;
     }
     @Override
     public boolean interpret(Context context) {
          if (experience >= context.getYearofExperience() &&
              context.getPermissibleGrades().contains(grade)){
                   return true;
              }
           return false;
     }
     @Override
     public String toString() {
          return "Experience: " + experience + ", grade: " +
                   grade;
     }
}
// OrExpression.java
class OrExpression implements Employee {
     private Employee emp1;
     private Employee emp2;
     public OrExpression(Employee emp1, Employee emp2) {
          this.emp1 = emp1;
          this.emp2 = emp2;
     }
     @Override



     public boolean interpret(Context context) {
          return emp1.interpret(context) ||
                  emp2.interpret(context);
     }
}
// AndExpression.java
class AndExpression implements Employee {
     private Employee emp1;
     private Employee emp2;
     public AndExpression(Employee emp1, Employee emp2) {
          this.emp1 = emp1;
          this.emp2 = emp2;
     }
     @Override
     public boolean interpret(Context context) {
          return emp1.interpret(context) &&
                  emp2.interpret(context);
     }
}
// NotExpression.java
class NotExpression implements Employee {
     private Employee emp;
     public NotExpression(Employee expr) {
          this.emp = expr;
     }



     @Override
```

```
     public boolean interpret(Context context) {
          return !emp.interpret(context);
     }
}
// InvalidExpression.java
class InvalidExpression implements Employee {
     @Override
     public boolean interpret(Context context) {
          return false; // result is always false
     }
}
// Context.java
import java.util.ArrayList;
import java.util.List;
class Context {
     private int yearofExperience;
     private List<String> permissibleGrades;
     public Context(int experience, String... allowedGrades) {
          this.yearofExperience = experience;
          this.permissibleGrades = new ArrayList<>();
          for (String grade : allowedGrades) {



               permissibleGrades.add(grade);
          }
     }
     public int getYearofExperience() {
          return yearofExperience;
     }
     public List<String> getPermissibleGrades() {
          return permissibleGrades;
     }
}
// ExpressionBuilder.java



class ExpressionBuilder {
     public Employee validateExpression(Employee emp1, String
                                        operator, Employee emp2) {
          // Converting the input into the lower case
          switch (operator.toLowerCase()) {
          case "or":
               return new OrExpression(emp1, emp2);
          case "and":
               return new AndExpression(emp1, emp2);
          case "not":
               return new NotExpression(emp1);
          default:
               System.out.println("You have used an invalid
                  operator:" + operator);
               System.out.println("(The result is always
                  false for this expression)");
               return new InvalidExpression();
          }
     }
     // Validating the rule: emp1 and (emp2 or (emp3 or emp4))
       public Employee validateComplexExpression(
                                        Employee emp1,
                                        Employee emp2,
                                        Employee emp3,
                                        Employee emp4
```

```
                                        ) {



    // (emp3 or emp4)
    Employee firstPhase = new OrExpression(emp3, emp4);
    // emp2 or (emp3 or emp4)
    Employee secondPhase = new OrExpression(emp2, firstPhase);
    // emp1 and (emp2 or (emp3 or emp4))
    Employee finalPhase = new AndExpression(emp1, secondPhase);
    return finalPhase;
   }
}
// Client.java
class Client {
     static Employee emp1, emp2, emp3, emp4;
     static Employee employee;
     static boolean result;
     static ExpressionBuilder builder = new
                                       ExpressionBuilder();
     // Minimum Criteria for promotion is:
     // The year of experience is minimum 10 years and



     // Employee grade should be either G2 or G3
     static Context context = new Context(10, "G2", "G3");
     public static void main(String[] args) {
          System.out.println("***Interpreter Pattern Demo***\n");
          initializeEmployees();
          validateSingleEmployee();
          validateSimpleRules();
          validateComplexRules();
     }
     private static void initializeEmployees() {
          emp1 = new IndividualEmployee(5, "G1");
          emp2 = new IndividualEmployee(10, "G2");
          emp3 = new IndividualEmployee(15, "G3");
          emp4 = new IndividualEmployee(20, "G4");
          System.out.println("Employee details are as follows:\n");
          System.out.println(emp1);
          System.out.println(emp2);
          System.out.println(emp3);
          System.out.println(emp4);
          System.out.println("---------");
     }



     private static void validateSingleEmployee() {
          System.out.println("Is emp1 eligible for promotion?
                              " + emp1.interpret(context));
          System.out.println("Is emp2 eligible for promotion?
                              " + emp2.interpret(context));
          System.out.println("Is emp3 eligible for promotion?
                              " + emp3.interpret(context));
          System.out.println("Is emp4 eligible for promotion?
                              " + emp4.interpret(context));
          System.out.println("---------");
     }
     private static void validateSimpleRules() {
          System.out.println("\nIs either emp1 or emp3
                                eligible for promotion?");
```

```
          employee = builder.validateExpression(emp1, "Or", emp3);
          result = employee.interpret(context);
          System.out.println(result);
          System.out.println("\nAre both emp2 and emp4
                              eligible for promotion?");
          employee = builder.validateExpression(emp2, "And", emp4);
          result = employee.interpret(context);
          System.out.println(result);
          System.out.println("\nIs the statement- 'emp3 is
                    NOT eligible for promotion' correct?");
          employee = builder.validateExpression(emp3, "Not", null);
          result = employee.interpret(context);
          System.out.println(result);
          System.out.println("---------");
     }



     private static void validateComplexRules() {
          // Validating the complex rule
          System.out.println("Are emp1 and any of emp2, emp3,
                           emp4 is eligible for promotion?");
          employee = builder.validateComplexExpression(
                                  emp1, emp2, emp3, emp4);
          result = employee.interpret(context);
          System.out.println(result);
          // Validating the complex rule
          System.out.println("Are emp2 and any of emp1, emp3,
                           emp4 is eligible for promotion?");
          employee = builder.validateComplexExpression(
                                     emp2, emp1, emp3, emp4);
          result = employee.interpret(context);
          System.out.println(result);
          System.out.println("---------");
     }
}
```

Output

Here's the output:

```
***Interpreter Pattern Demo***
Employee details are as follows:
Experience: 5, grade: G1



Experience: 10, grade: G2
Experience: 15, grade: G3
Experience: 20, grade: G4
---------
Is emp1 eligible for promotion? false
Is emp2 eligible for promotion? true
Is emp3 eligible for promotion? true
Is emp4 eligible for promotion? false
---------
Is either emp1 or emp3 eligible for promotion?
true
Are both emp2 and emp4 eligible for promotion?
false
Is the statement- 'emp3 is NOT eligible for promotion' correct?
```

```
false
---------
Are emp1 and any of emp2, emp3, emp4 is eligible for promotion?
false



Are emp2 and any of emp1, emp3, emp4 is eligible for promotion?
true
---------
```

Analysis

In this implementation, I evaluate many different expressions. For your easy understanding, I categorize them in different methods where each of them is used to evaluate similar kinds of expressions.

Q&A Session

25.1 When should you use this pattern?

In daily programming life, to be honest, it is not needed much. However, in some rare situations, you may need to work with your own programming language where it could come in handy. But before you proceed, you must ask yourself, what is the Return on Investment (ROI)?

25.2 What are the advantages of using the Interpreter design pattern?

You can have the following advantages:

- You are very much involved in the process of how to define grammar for your language and how to represent and interpret those sentences. You can change and extend the grammar also.
- You have full freedom over how to interpret these expressions.

25.3 What are the challenges associated with using the Interpreter design pattern?

I believe that the amount of work is the biggest concern. Also, maintaining complex grammar becomes tricky because you need to create (and maintain) separate classes to deal with different rules.

25.4 In demonstration 1, when you evaluated a complex expression, you passed four arguments inside the method `validateComplexExpression(…)`. How should I proceed if I need to parse a variable number of arguments?

It's a sample demonstration to give you an idea about the Interpreter design pattern. Let's consider an alternative implementation, which is as follows.

Alternative Implementation

Here I present an alternative demonstration. If you know how to evaluate postfix notations, it will be very easy for you to understand this imple-

mentation; otherwise, let's have a quick discussion on this.

You may have seen three different notations for a mathematical expression, which are as follows:

- **Infix** notation is the common arithmetic and logical formula notation in which operators are written between the operands such as 5+2.
- **Postfix** (a.k.a. Reverse Polish notation) is a mathematical notation in which every operator follows all of its operands such as 52+.
- **Prefix** (Polish notation) operators appear to the left of their operands such as +52.

The infix notation is common in mathematical expressions. The other two notations can be used for interpreters of programming languages.

Demonstration 2

In the upcoming implementation, I changed `ExpressionBuilder.java`. This means I had to rewrite the client code accordingly. There is no change in the remaining parts of demonstration 1. All parts of the program are inside the package `jdp3e.interpreter.implementation_2`.

Let's have a look at this new `EmployeeBuilder.java`. Here are the key characteristics:

- All the valid employees are initialized inside the constructor.
- There is a `parse(…)` method inside this class. When it sees the string `emp1` in input, it pushes the `emp1` instance inside a stack. It works similarly for `emp2`, `emp3`, or `emp4`. Otherwise, it assumes that it gets an operator.
- When it sees a binary operator (the `and` operator and `or` operator), it pops up the top two elements from the stack, evaluates the complex expression, and pushes back the result on the stack. You understand that for a unary operator such as `not`, you need to pop only the top element from the stack and proceed.
- If the operator is other than `or`, `and`, or `not`, it throws an exception.
- You use the `toLowercase()` method to convert these words into lowercase before you evaluate a complex expression. As a result, a client need not worry about the case sensitivity of the following words: 'and', 'or', and 'not'.

Here is the `EmployeeBuilder` class:

```
// This class can parse the input
class EmployeeBuilder {
     Employee emp1, emp2, emp3, emp4;
     Employee tempEmp;// Used as a temporary variable
     public EmployeeBuilder() {
          // Initialize valid Employees
          emp1 = new IndividualEmployee(5, "G1");
          emp2 = new IndividualEmployee(10, "G2");
          emp3 = new IndividualEmployee(15, "G3");
          emp4 = new IndividualEmployee(20, "G4");
     }
     Stack<Employee> currentStack = new Stack<>();
     public void parse(String input) {
```

```
          String[] tokens = input.split(" ");
          for (String token : tokens) {
               if (token.equals("emp1"))
                    currentStack.push(emp1);
               else if (token.equals("emp2"))
                    currentStack.push(emp2);
               else if (token.equals("emp3"))
                    currentStack.push(emp3);
               else if (token.equals("emp4"))
                    currentStack.push(emp4);
               // Got an operator
               else {
                    Employee emp1, emp2 = null;
                    emp1 = currentStack.pop();
                    // Expression 2 is not needed for a
                      // 'not' operator
                    if (token.equals("and")||
                          token.equals("or") ) {
                         emp2 = currentStack.pop();
                    }
                    tempEmp = evaluate(token, emp1, emp2);
                    currentStack.push(tempEmp);
               }


          }
     }
     private Employee evaluate(String operator, Employee emp1,
                               Employee emp2) {
          switch (operator.toLowerCase()) {
          case "and":
               tempEmp = new AndExpression(emp1, emp2);
               break;
          case "or":
               tempEmp = new OrExpression(emp1, emp2);
               break;
          case "not":
               tempEmp = new NotExpression(emp1);
               break;
          default:
               throw new IllegalArgumentException("Invalid
                                    operator: " + operator);
          }
          return tempEmp;
     }
}
```

Now let's look into the client class. Notice that this time the input follows the postfix notations. Additional expressions (that are commented out now) are supplied for your reference so that you can uncomment and verify the final value of these expressions.

```
class Client {
     public static void main(String[] args) {
          System.out.println("***Modified Interpreter Pattern
                              Demonstration-2***\n");
          // Minimum Criteria for promotion is:
          // The year of experience is a minimum of 10 yrs.
           // and Employee grade should be
           // either G2 or G3


          Context context = new Context(10, "G2", "G3");
```

```
            // String input="emp1";//false
            // String input="emp1 not";//true
            // String input="emp2 not";//false
            // String input="emp4 emp2 not and";//false
            // String input="emp3 emp4 emp2 not and or";//true
            String input = "emp2 emp3 emp4 emp2 not and or
                            or";// true
            EmployeeBuilder builder = new EmployeeBuilder();
            builder.parse(input);
            Employee finalExpression =
                            builder.currentStack.pop();
            Boolean result =
                            finalExpression.interpret(context);
            System.out.print(input + ":" + result);
            // Removing additional elements(if any)
            builder.currentStack.clear();
        }
}
```

Output

When you run this modified program, you'll see the following output:

```
***Modified Interpreter Pattern Demonstration-2***
emp2 emp3 emp4 emp2 not and or or:true
```

That's the end of part II of the book. I hope you enjoyed the detailed implementations of the GoF patterns. Now you can move to the next part of the book, which covers a few more interesting patterns.

Previous chapter
< 24. Visitor Pattern

Next chapter
Part III. Additional Design Patterns >